Got it — you've listed a set of files that look like they belong to a **Next.js + TypeScript + Tailwind CSS + Prisma + i18n** project, with authentication and feature-specific services.

I'll give you a **clear, runnable boilerplate structure** for each file so you can integrate them into your project.

This will be **complete**, **TypeScript-ready**, and **aligned with Next.js best practices**.

1. `package.json`

```
{
  "name": "nextjs-ts-tailwind-prisma-i18n-app",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "next lint",
    "prisma": "prisma",
    "seed": "ts-node prisma/prisma-seed.ts"
  },
  "dependencies": {
    "next": "13.5.0",
    "react": "18.2.0",
    "react-dom": "18.2.0",
    "tailwindcss": "^3.3.0",
    "autoprefixer": "^10.4.0",
    "postcss": "^8.4.0",
    "@prisma/client": "^5.0.0",
    "bcrypt": "^5.1.0",
    "next-i18next": "^13.0.0",
    "zod": "^3.22.0"
  },
  "devDependencies": {
    "typescript": "^5.2.0",
    "@types/node": "^20.4.0",
    "@types/react": "^18.2.0",
    "prisma": "^5.0.0",
    "ts-node": "^10.9.0"
```

```
  }
}
```

2. tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2020",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": false,
    "skipLibCheck": true,
    "strict": true,
    "forceConsistentCasingInFileNames": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "ESNext",
    "moduleResolution": "Node",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true
  },
  "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx"],
  "exclude": ["node_modules"]
}
```

3. next.config.js

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  reactStrictMode: true,
  swcMinify: true,
  i18n: {
    locales: ["en", "fr"],
    defaultLocale: "en"
  }
};
```

```
module.exports = nextConfig;
```

4. tailwind.config.ts

```
import type { Config } from "tailwindcss";

const config: Config = {
  content: [
    "./pages/**/*.{js,ts,jsx,tsx}",
    "./components/**/*.{js,ts,jsx,tsx}",
    "./app/**/*.{js,ts,jsx,tsx}"
  ],
  theme: {
    extend: {}
  },
  plugins: []
};

export default config;
```

5. i18n.config.ts

```
import { NextI18NextConfig } from "next-i18next";

const i18nConfig: NextI18NextConfig = {
  defaultLocale: "en",
  locales: ["en", "fr"],
  localePath: "./public/locales"
};

export default i18nConfig;
```

6. middleware.ts

```
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";
```

```
export function middleware(request: NextRequest) {
  const token = request.cookies.get("auth_token");
  if (!token && request.nextUrl.pathname.startsWith("/dashboard")) {
    return NextResponse.redirect(new URL("/login", request.url));
  }
  return NextResponse.next();
}

export const config = {
  matcher: ["/dashboard/:path*"]
};
```

7. schema.prisma

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id        Int      @id @default(autoincrement())
  email     String   @unique
  password  String
  createdAt DateTime @default(now())
}

model Product {
  id    Int    @id @default(autoincrement())
  name  String
  price Float
}
```

8. prisma-seed.ts

```
import { PrismaClient } from "@prisma/client";
import bcrypt from "bcrypt";

const prisma = new PrismaClient();

async function main() {
  const hashedPassword = await bcrypt.hash("admin123", 10);
  await prisma.user.create({
    data: {
      email: "admin@example.com",
      password: hashedPassword
    }
  });
  console.log("Database seeded successfully.");
}

main()
  .catch((e) => {
    console.error(e);
    process.exit(1);
  })
  .finally(async () => {
    await prisma.$disconnect();
  });
```

9. lib-auth.ts

```
import bcrypt from "bcrypt";
import { PrismaClient } from "@prisma/client";

const prisma = new PrismaClient();

export async function authenticate(email: string, password: string) {
  const user = await prisma.user.findUnique({ where: { email } });
  if (!user) return null;
  const isValid = await bcrypt.compare(password, user.password);
  return isValid ? user : null;
}
```

10. app-api-auth-route.ts

```
import { NextApiRequest, NextApiResponse } from "next";
import { authenticate } from "../lib-auth";

export default async function handler(req: NextApiRequest, res:
NextApiResponse) {
  if (req.method !== "POST") {
    return res.status(405).json({ message: "Method not allowed" });
  }
  const { email, password } = req.body;
  const user = await authenticate(email, password);
  if (!user) {
    return res.status(401).json({ message: "Invalid credentials" });
  }
  res.status(200).json({ message: "Login successful", user });
}
```

11–14. Feature Services (features-sales-service.ts, features-products-service.ts, features-customers-service.ts, features-exp.ts)

Example for features-products-service.ts:

```
import { PrismaClient } from "@prisma/client";

const prisma = new PrismaClient();

export async function getAllProducts() {
  return prisma.product.findMany();
}

export async function createProduct(name: string, price: number) {
  return prisma.product.create({
    data: { name, price }
  });
}
```

If you want, I can **bundle all these into a ready-to-run Next.js project folder structure** so you can just npm install and start coding.

Do you want me to prepare that full folder layout next?